

Օբյեկտային ուղղվածությամբ ծրագրավորում C++ լեզվում

Նյութը ներկայացնում է օբյեկտային ուղղվածությամբ ծրագրավորման առաջին հիմնական հասկացությունները: Դասի ավարտին ուսանողը պետք է կարողանա սահմանել պարզ class, ստեղծել օբյեկտներ, տարբերել տվյալների անդամներն ու մեթոդները, կիրառել private, public և protected բաժինները և կազմակերպել օբյեկտի տվյալների անվտանգ փոփոխումը մեթոդների միջոցով:

1 Օբյեկտային ուղղվածությամբ ծրագրավորման գաղափարը

Թելադրելու նյութ: Օբյեկտային ուղղվածությամբ ծրագրավորումը ծրագիրը կազմակերպում է օբյեկտների շուրջ: Յուրաքանչյուր օբյեկտ ունի վիճակ և վարքագիծ: Վիճակը ներկայացվում է տվյալներով, իսկ վարքագիծը՝ այդ տվյալների հետ աշխատող մեթոդներով: Այս մոտեցումը թույլ է տալիս նույն տեղում պահել փոխկապակցված տվյալներն ու դրանց մշակման կանոնները:

Բացատրություն: Օրինակ՝ բանկային հաշվի վիճակը կազմում են հաշվի համարը և մնացորդը: Վարքագիծը ներկայացնում են գումարի մուտքագրման, ելքագրման և մնացորդի ստացման գործողությունները: Եթե մնացորդը հասանելի լինի միայն մեթոդների միջոցով, դասը կարող է արգելել բացասական գումարների մուտքագրումն ու թույլատրելից մեծ ելքագրումը:

2 Դաս և օբյեկտ

Դասը օգտատիրոջ կողմից սահմանվող տիպ է: Այն նկարագրում է, թե ինչ տվյալներ և գործողություններ պետք է ունենան տվյալ տիպի օբյեկտները: Օբյեկտը դասի հիման վրա ստեղծված կոնկրետ փոփոխական է: Դասը կարելի է դիտարկել որպես ընդհանուր նկարագրություն, իսկ օբյեկտը՝ այդ նկարագրությամբ ստեղծված կոնկրետ արժեք:

```
class Student {  
public:  
    string name;  
    int course;  
};  
  
Student firstStudent;  
Student secondStudent;
```

Student-ը դասի անունն է: firstStudent և secondStudent փոփոխականները երկու տարբեր օբյեկտներ են: Յուրաքանչյուր օբյեկտ ունի name և course դաշտերի սեփական արժեքները: Մեկ օբյեկտի փոփոխությունը չի փոխում մյուս օբյեկտի վիճակը:

3 Դասի ընդհանուր շարահյուսությունը

```
class ClassName {  
private:  
    // փակ տվյալներ և մեթոդներ  
  
public:  
    // բաց մեթոդներ  
  
protected:  
    // ժառանգ դասերի համար հասանելի անդամներ  
};
```

class բանալի բառից հետո գրվում է դասի անունը: Դասի մարմինը սահմանափակվում է ձևավոր փակագծերով, իսկ նկարագրությունից հետո դրվում է կետ-ստորակետ: private, public և protected բառերը բաժինների սկիզբն են ցույց տալիս և ավարտվում են երկու կետով: Նույն բաժինը կարող է պարունակել մի քանի անդամ:

4 Դասի բաղադրիչները

Տվյալների անդամներ

Տվյալների անդամները նկարագրում են օբյեկտի վիճակը: Դրանք կարող են ունենալ C++ լեզվի ցանկացած թույլատրելի տիպ: Օրինակ՝ Product դասի name, price և quantity անդամները ներկայացնում են մեկ ապրանքի տվյալները: Յուրաքանչյուր օբյեկտ սովորաբար ունի ոչ static տվյալների անդամների սեփական պատճենը:

Անդամ մեթոդներ

Անդամ մեթոդները դասի ներսում հայտարարված ֆունկցիաներ են: Դրանք աշխատում են տվյալ օբյեկտի անդամների հետ և կանչվում են կետ գործողությամբ: Մեթոդը կարող է ստանալ պարամետրեր, վերադարձնել արժեք և ստուգել գործողության թույլատրելիությունը:

```
class Product {  
private:  
    string name;  
    double price = 0;  
    int quantity = 0;  
  
public:  
    void setPrice(double value) {  
        if (value >= 0) price = value;  
    }  
  
    double totalValue() const {  
        return price * quantity;  
    }  
};
```

5 Հասանելիության բաժինները

Բաղադրիչ	Տեսանելիություն	Նպատակ	Օրինակ
private	միայն դասի ներսում	տվյալների պաշտպանություն	balance, grade
public	ծրագրի հասանելի մասերում	օբյեկտի կառավարում	deposit(), print()
protected	դասում և ժառանգ դասերում	ժառանգման համար հասանելի անդամներ	baseValue

private անդամին հնարավոր է դիմել միայն տվյալ դասի մեթոդներից կամ հատուկ թույլտվություն ունեցող ֆունկցիաներից ու դասերից: public անդամները կազմում են դասի արտաքին միջերեսը: protected բաժինը հիմնականում օգտագործվում է ժառանգման ժամանակ և այս դասում ներկայացվում է ընդհանուր գաղափարի մակարդակով:

Եթե class-ի սկզբում բաժին նշված չէ, անդամները լռելյայն private են: struct-ի անդամները լռելյայն public են: Երկու ձևերն էլ կարող են ունենալ տվյալներ և մեթոդներ, սակայն class-ը հաճախ կիրառվում է տվյալները փակ պահելու և վերահսկվող միջերես տրամադրելու համար:

6 Օբյեկտի ստեղծում և մեթոդի կանչ

```
Product keyboard;
keyboard.setPrice(12000);

Product mouse;
mouse.setPrice(6500);

cout << keyboard.totalValue();
```

Կետ գործողության ձախ կողմում գրվում է օբյեկտի անունը, աջ կողմում՝ public դաշտը կամ մեթոդը: private դաշտին main ֆունկցիայից ուղիղ դիմելը կոմպիլյացիոն սխալ է: Օբյեկտի տվյալները պետք է փոփոխել դասի public մեթոդների միջոցով:

7 Մեթոդի նկարագրումը դասի ներսում և դրսում

Մեթոդը դասի ներսում

```
class Rectangle {
private:
    double width = 0;
    double height = 0;
public:
    double area() const {
        return width * height;
    }
};
```

Մեթոդը դասից դուրս

```
class Rectangle {
private:
    double width = 0;
    double height = 0;
public:
    double area() const;
};

double Rectangle::area() const {
    return width * height;
}
```

Դասից դուրս նկարագրելիս կիրառվում է տեսանելիության տիրույթի :: գործողությունը: Rectangle::area գրառումը ցույց է տալիս, որ area մեթոդը պատկանում է Rectangle դասին: Հայտարարությունն ու նկարագրությունը պետք է ունենան նույն վերադարձվող տիպը, պարամետրերը և const նշումը:

8 Տվյալների թաքցում և վերահսկվող փոփոխում

Տվյալների թաքցումը նշանակում է, որ ծրագրի արտաքին հատվածը չի փոխում օբյեկտի ներքին վիճակը անվերահսկելի ձևով: Դասը public մեթոդների միջոցով ստուգում է նոր արժեքը և պահպանում օբյեկտի վավեր վիճակը: Այս սկզբունքը ինկապսուլյացիայի հիմնական կիրառություններից է:

```
class Student {
private:
    string name;
    double grade = 0;
public:
    void setName(const string& value) {
        if (!value.empty()) name = value;
    }

    bool setGrade(double value) {
        if (value < 0 || value > 20) return false;
        grade = value;
        return true;
    }

    const string& getName() const { return name; }
    double getGrade() const { return grade; }
};
```

Setter մեթոդը փոփոխում է դաշտը և անհրաժեշտության դեպքում ստուգում արժեքը: Getter մեթոդը վերադարձնում է դաշտի արժեքը: Ամեն դաշտի համար մեխանիկորեն getter և setter ստեղծելը պարտադիր չէ: Մեթոդը պետք է արտահայտի դասի իրական թույլատրելի գործողությունը:

9 const անդամ մեթոդ

Մեթոդի պարամետրերի փակագծերից հետո գրված const-ը նշանակում է, որ մեթոդը չի փոխում օբյեկտի ոչ mutable տվյալների անդամները: Միայն ընթերցող մեթոդները

Նպատակահարմար է նշել const, քանի որ այդ դեպքում դրանք կարելի է կանչել նաև const օբյեկտների համար:

```
double getBalance() const {
    return balance;
}

void printAccount(const BankAccount& account) {
    cout << account.getBalance();
}
```

10 this ցուցիչը

Յուրաքանչյուր ոչ static անդամ մեթոդ ունի ընթացիկ օբյեկտի հասցեն պահող թաքնված this ցուցիչ: Այն օգտակար է, երբ պարամետրի անունը համընկնում է դաշտի անվան հետ: this->name-ը նշանակում է ընթացիկ օբյեկտի name դաշտը, իսկ աջ կողմի name-ը մեթոդի պարամետրն է:

```
void setName(const string& name) {
    if (!name.empty()) {
        this->name = name;
    }
}
```

11 Օբյեկտի փոխանցումը \$ուսկցիային

```
void printStudent(const Student& student) {
    cout << student.getName() << " "
        << student.getGrade();
}

Student betterStudent(Student first, Student second) {
    if (first.getGrade() >= second.getGrade()) return first;
    return second;
}
```

Արժեքով փոխանցման դեպքում \$ուսկցիան ստանում է օբյեկտի պատճենը: Հաստատուն հղումը չի ստեղծում պատճեն և չի թույլատրում փոփոխել օբյեկտը: Միայն ընթերցելու համար սովորաբար ընտրվում է const հղումը: \$ուսկցիան կարող է նաև վերադարձնել դասի օբյեկտ:

12 Ամբողջական օրինակ BankAccount դասը

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance = 0;
```

```

public:
    void setOwner(const string& value) {
        if (!value.empty()) owner = value;
    }

    bool deposit(double amount) {
        if (amount <= 0) return false;
        balance += amount;
        return true;
    }

    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    const string& getOwner() const { return owner; }
    double getBalance() const { return balance; }
};

int main() {
    BankAccount account;
    account.setOwner("Ani");
    account.deposit(50000);
    if (!account.withdraw(12000))
        cout << "Operation failed\n";

    cout << account.getOwner() << " "
        << account.getBalance();
    return 0;
}

```

Օրինակում balance դաշտը փակ է: deposit և withdraw մեթոդները պաշտպանում են մնացորդը սխալ փոփոխություններից: main ֆունկցիան աշխատում է միայն public միջերեսի հետ և կարիք չունի իմանալու, թե դասը ներսում ինչպես է պահպանում տվյալները:

13 Տարածված սխալներ

1. class նկարագրությունից հետո կետ-ստորակետը բաց թողնելը:
2. private դաշտին main ֆունկցիայից ուղիղ դիմելը:
3. Հասանելիության բաժնի անունից հետո երկու կետը բաց թողնելը:
4. Օբյեկտի մեթոդը կանչելիս փակագծերը չգրելը:
5. Դասից դուրս մեթոդ նկարագրելիս ClassName:: մասը չգրելը:
6. Միայն ընթերցող մեթոդը const չնշելը:
7. Setter-ում մուտքային արժեքները չստուգելը:
8. Մեկ օբյեկտի փոփոխությունը բոլոր օբյեկտների վրա տարածվելու սխալ ենթադրությունը:

14 Դասի ընթացքում փոքր գործնական աշխատանք

Սահմանել Rectangle դաս՝ private width և height դաշտերով: Ստեղծել setDimensions մեթոդ, որը ընդունում է միայն դրական արժեքներ, ինչպես նաև area և perimeter const մեթոդներ: main ֆունկցիայում ստեղծել երկու օբյեկտ, մուտքագրել դրանց չափերը և արտածել ավելի մեծ մակերես ունեցող ուղղանկյունը:

15 Ամփոփիչ հարցեր

1. Ի՞նչ է դասը և ինչո՞վ է այն տարբերվում օբյեկտից:
2. Որո՞նք են դասի երկու հիմնական տեսակի անդամները:
3. Ի՞նչ տարբերություն կա private և public բաժինների միջև:
4. Ո՞ր հասանելիությունն է լռելյայն class-ի ներսում:
5. Ինչպե՞ս է կանչվում օբյեկտի public մեթոդը:
6. Ինչո՞ւ է օգտակար դաշտերը փակ պահելը:
7. Ի՞նչ է ցույց տալիս :: գործողությունը մեթոդի նկարագրության մեջ:
8. Ի՞նչ է նշանակում մեթոդի վերջում գրված const-ը:
9. Ե՞րբ է կիրառվում this ցուցիչը:
10. Ինչո՞ւ է օբյեկտը միայն ընթերցող ֆունկցիային փոխանցվում const հղումով: